

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống tường lửa kết hợp thông minh
dựa trên công nghệ XDP/eBPF**

LÊ NAM PHONG

Phong.LN225149@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin - Kỹ thuật máy tính

Giảng viên hướng dẫn: ThS. Nguyễn Quốc Khánh

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống tường lửa kết hợp thông minh
dựa trên công nghệ XDP/eBPF**

LÊ NAM PHONG

Phong.LN225149@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin - Kỹ thuật máy tính

Giảng viên hướng dẫn: ThS. Nguyễn Quốc Khánh

Chữ kí GVHD

Khoa:

Kỹ thuật máy tính

Trường:

Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Lời cảm ơn của sinh viên (SV) tới người yêu, gia đình, bạn bè, thầy cô, và chính bản thân mình vì đã chăm chỉ và quyết tâm thực hiện ĐATN để đạt kết quả tốt nhất, nên viết phần cảm ơn ngắn gọn, tránh dùng các từ sáo rỗng, giới hạn trong khoảng 100-150 từ.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Sinh viên viết tóm tắt ĐATN của mình trong mục này, với 200 đến 350 từ. Theo trình tự, các nội dung tóm tắt cần có: (i) Giới thiệu vấn đề (tại sao có vấn đề đó, hiện tại được giải quyết chưa, có những hướng tiếp cận nào, các hướng này giải quyết như thế nào, hạn chế là gì), (ii) Hướng tiếp cận sinh viên lựa chọn là gì, vì sao chọn hướng đó, (iii) Tổng quan giải pháp của sinh viên theo hướng tiếp cận đã chọn, và (iv) Đóng góp chính của ĐATN là gì, kết quả đạt được sau cùng là gì. Sinh viên cần viết thành đoạn văn, không được viết ý hoặc gạch đầu dòng.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

Mục này khuyến khích sinh viên viết lại mục “Tóm tắt” đề án tốt nghiệp ở trang trước bằng tiếng Anh. Phần này phải có đầy đủ các nội dung như trong phần tóm tắt bằng tiếng Việt. Sinh viên không nhất thiết phải trình bày mục này.

Nhưng nếu lựa chọn trình bày, sinh viên cần đảm bảo câu từ và ngữ pháp chuẩn tắc, nếu không sẽ có tác dụng ngược, gây phản cảm.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Các giải pháp hiện tại và hạn chế	1
1.3 Mục tiêu và định hướng giải pháp	1
1.4 Đóng góp của đề án	2
1.5 Bố cục đề án	2
CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT	4
2.1 Ngữ cảnh của bài toán.....	4
2.2 Các kết quả nghiên cứu tương tự	4
2.3 Vòng đời của một gói tin trong Linux Networking Stack.....	4
2.4 eBPF	6
2.4.1 Lịch sử và tổng quan	6
2.4.2 Cơ chế hoạt động và vòng đời của chương trình eBPF	6
2.4.3 BPF Maps	8
2.5 XDP	9
2.5.1 Tổng quan và vị trí trong kernel.....	9
2.5.2 Các chế độ hoạt động	9
2.5.3 Các return code.....	10
2.6 Firewall phân tầng	10
2.6.1 Lọc gói tin tầng 3, tầng 4	10
2.6.2 Lọc trạng thái tầng 4	11
2.6.3 Kiểm tra gói tin tầng 7.....	11
2.7 Các dạng tấn công từ chối dịch vụ	12
2.7.1 Tấn công SYN Flood.....	12

2.7.2 Tấn công UDP Flood.....	12
2.7.3 Tấn công HTTP Flood	13
2.7.4 Slowloris	13
2.7.5 IP Spoofing và Random Source Flooding	13
2.8 Random Early Detection (RED) và Weighted RED (WRED)	14
2.8.1 Cơ chế Random Early Detection (RED)	14
2.8.2 Weighted RED và ưu tiên theo loại traffic	14
2.8.3 Ứng dụng Probabilistic Drop trong môi trường DDoS	15
2.9 GeoIP, ASN và định dạng MMDB	15
2.9.1 Khái niệm GeoIP và ASN	15
2.9.2 Định dạng cơ sở dữ liệu MaxMind DB (MMDB)	16
2.10 Iptables và Nftables.....	16
2.10.1 Kiến trúc Iptables và Netfilter	16
2.10.2 Sự chuyển dịch sang Nftables	16
2.11 Suricata	17
2.11.1 Tổng quan về Suricata.....	17
2.11.2 Vai trò trong kiểm tra gói tin sâu (DPI).....	17
2.12 Connection Tracking (Conntrack).....	17
2.12.1 Nguyên lý hoạt động.....	17
2.12.2 Conntrack trong các hệ thống hiệu năng cao	17
2.13 Tổng kết chương	18
CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT.....	19
3.1 Tổng quan giải pháp.....	19
3.2 Phát biểu bài toán	19
3.3 Kiến trúc Hybrid Firewall tổng thể.....	19
3.4 Luồng xử lý gói tin	19

3.5 Thiết kế Tier 1 — Adaptive Mitigation	19
3.5.1 Tổng quan Tier 1	19
3.5.2 geo_trust_map	19
3.5.3 trusted_map.....	19
3.5.4 mitigation_map.....	19
3.5.5 Cơ chế Probabilistic Drop	19
3.6 Thiết kế Tier 2 — Per-IP Defense.....	19
3.6.1 Tổng quan Tier 2.....	19
3.6.2 rate_limit_map.....	19
3.6.3 auto_block_map.....	19
3.6.4 subnet_map và rule_map	19
3.7 Watcher Daemon và cơ chế Adaptive.....	19
3.7.1 Kiến trúc Watcher	19
3.7.2 Telemetry và Sliding Window	19
3.7.3 Hysteresis — Escalation và De-escalation	19
3.7.4 Tích hợp tín hiệu từ Suricata và Iptables.....	19
3.7.5 Cơ chế Manual Override.....	19
3.8 Control Plane	19
3.8.1 Go Controller	19
3.8.2 Conntrack Listener và trusted_map.....	19
3.8.3 Admin Seed và geo_trust_map	19
3.8.4 Thiết kế REST API	19
3.9 Nguyên tắc thiết kế	19
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG.....	20
4.1 Môi trường triển khai	20
4.1.1 Cấu hình phần cứng.....	20

4.1.2 Cấu hình phần mềm	20
4.1.3 Mô hình máy ảo	20
4.2 Cấu trúc mã nguồn.....	20
4.2.1 Tổng quan cấu trúc thư mục	20
4.2.2 Phần kế thừa và phần mở rộng.....	20
4.3 Các thành phần chính.....	20
4.3.1 XDP Program	20
4.3.2 Go Controller	20
4.3.3 REST API	20
4.3.4 Web UI	20
4.4 Các chức năng chính	20
4.4.1 Quản lý Rule	20
4.4.2 Quản lý Tier 1.....	20
4.4.3 Quản lý Tier 2.....	20
4.4.4 Theo dõi Telemetry	20
4.5 Kết quả triển khai	20
CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ	21
5.1 Mô hình thực nghiệm.....	21
5.1.1 Mô hình hệ thống.....	21
5.1.2 Công cụ và phương pháp đo lường.....	21
5.1.3 Các tham số đánh giá	21
5.2 Kịch bản Baseline.....	21
5.3 Kịch bản SYN Flood với IP cố định.....	21
5.4 Kịch bản Random Source Flood.....	21
5.5 Kịch bản HTTP Application Layer Attack.....	21

5.6 Phân tích kết quả	21
5.6.1 Hiệu quả của Tier 1 với Random Source Flood	21
5.6.2 Hiệu quả của Tier 2 với IP cố định.....	21
5.6.3 Vai trò của Geo Trust.....	21
5.6.4 Vai trò của Suricata và Iptables	21
5.7 Hạn chế của hệ thống	21
CHƯƠNG 6. KẾT LUẬN	22
6.1 Kết luận.....	22
6.2 Hướng phát triển trong tương lai	22
TÀI LIỆU THAM KHẢO.....	26
PHỤ LỤC.....	28
A. HƯỚNG DẪN VIẾT ĐATN	28
A.1 Quy định chung	28
A.2 Ngành học.....	29
A.3 Đánh dấu (bullet) và đánh số (numering)	29
A.4 Cách thêm bảng	30
A.5 Chèn hình ảnh	30
A.6 Tài liệu tham khảo	31
A.7 Cách viết phương trình và công thức toán học.....	31
A.8 Qui cách đóng quyển.....	31
B. ĐẶC TẢ USE CASE.....	33
B.1 Đặc tả use case “Thông kê tình hình mượn sách”	33
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	33

DANH MỤC HÌNH VẼ

Hình A.1	Internet vạn vật	30
Hình A.2	Qui cách đóng quyển đồ án	32
Hình A.3	Qui cách đóng quyển đồ án	32

DANH MỤC BẢNG BIỂU

Bảng A.1	Table to test captions and labels.	30
----------	--	----

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
EUD	Phát triển ứng dụng người dùng cuối(End-User Development)
GWT	Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IaaS	Dịch vụ hạ tầng

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Lưu ý: Mẫu ĐATN này được thiết kế phù hợp với đồ án tốt nghiệp theo hướng nghiên cứu. Mẫu đề tài này là gợi ý tham khảo. Tùy từng đề tài, cấu trúc có thể thay đổi ít nhiều. Sinh viên cần tham khảo ý kiến của giáo viên hướng dẫn để đưa ra cấu trúc hợp lý nhất cho đề tài của mình.

Trước khi viết ĐATN, sinh viên cần đọc kỹ hướng dẫn và quy định chi tiết về cách viết ĐATN trong Phụ lục A.

Khi đóng quyển ĐATN, sinh viên cần lưu ý tuân thủ hướng dẫn ở phụ lục A.9

SV cần đặc biệt lưu ý cách hành văn. Mỗi đoạn văn không được quá dài và cần có ý tứ rõ ràng, bao gồm duy nhất một ý chính và các ý phân tích bổ trợ để làm rõ hơn ý chính. Các câu văn trong đoạn phải đầy đủ chủ ngữ vị ngữ, cùng hướng đến chủ đề chung. Câu sau phải liên kết với câu trước, đoạn sau liên kết với đoạn trước. Trong văn phong khoa học, sinh viên không được dùng từ trong văn nói, không dùng các từ phóng đại, thái quá, các từ thiếu khách quan, thiên về cảm xúc, về quan điểm cá nhân như “tuyệt vời”, “cực hay”, “cực kỳ hữu ích”, v.v. Các câu văn cần được tối ưu hóa, đảm bảo rất khó để thể thêm hoặc bớt đi được dù chỉ một từ. Cách diễn đạt cần ngắn gọn, súc tích, không dài dòng.

Chương 1 có độ dài từ 3 đến 6 trang với các nội dung sau đây

1.1 Đặt vấn đề

Phần này sinh viên mô tả bài toán cần giải quyết, lý do tại sao lại chọn bài toán đó, ý nghĩa/tầm quan trọng của bài toán.

Tiêu đề của chương này có thể để là “đặt vấn đề”, hoặc lấy chính tên của bài toán mà sinh viên định giải quyết, ví dụ có thể đặt tiêu đề là “Bài toán dự đoán...”

1.2 Các giải pháp hiện tại và hạn chế

Sinh viên trước tiên cần trình bày tổng quan các kết quả của các nghiên cứu hiện nay cho bài toán giới thiệu ở phần 1.1. Sau đấy, sinh viên đưa ra các hạn chế của các giải pháp hiện tại.

1.3 Mục tiêu và định hướng giải pháp

Trong chương này, sinh viên trước hết trình bày mục tiêu của đồ án là gì, sau đấy sinh viên đề xuất định hướng giải pháp của mình. Tốt nhất là với trình bày từng giải pháp đối với mỗi vấn đề nêu ra trong chương 1.2.

1.4 Đóng góp của đề án

Trong phần này sinh viên liệt kê cụ thể, ngắn gọn các đóng góp của đề án. Ví dụ:

Đề án này có 3 đóng góp chính như sau:

1. Đề án đề xuất một phương pháp tiền xử lý dữ liệu nhằm loại bỏ nhiễu và dữ liệu ngoại lai trước khi đưa vào huấn luyện mô hình.
2. ...
3. ...

1.5 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày về v.v.

Trong Chương 3, em/tôi giới thiệu về v.v.

Chú ý: Sinh viên cần viết mô tả thành đoạn văn đầy đủ về nội dung chương. Tuyệt đối không viết ý hay gạch đầu dòng. Chương 1 không cần mô tả trong phần này.

Ví dụ tham khảo mô tả chương trong phần bố cục đề án tốt nghiệp: Chương *** trình bày đóng góp chính của đề án, đó là một nền tảng ABC cho phép khai phá và tích hợp nhiều nguồn dữ liệu, trong đó mỗi nguồn dữ liệu lại có định dạng đặc thù riêng. Nền tảng ABC được phát triển dựa trên khái niệm DEF, là các module ngữ nghĩa trợ giúp người dùng tìm kiếm, tích hợp và hiển thị trực quan dữ liệu theo mô hình cộng tác và mô hình phân tán.

Chú ý: Trong phần nội dung chính, mỗi chương của đề án nên có phần Tổng quan và Kết chương. Hai phần này đều có định dạng văn bản “Normal”, sinh viên không cần tạo định dạng riêng, ví dụ như không in đậm/in nghiêng, không đóng khung, v.v...

Trong phần Tổng quan của chương N, sinh viên nên có sự liên kết với chương N-1 rồi trình bày sơ qua lý do có mặt của chương N và sự cần thiết của chương này trong đề án. Sau đó giới thiệu những vấn đề sẽ trình bày trong chương này là gì, trong các đề mục lớn nào.

Ví dụ về phần Tổng quan: Chương 3 đã thảo luận về nguồn gốc ra đời, cơ sở lý thuyết và các nhiệm vụ chính của bài toán tích hợp dữ liệu. Chương 4 này sẽ trình bày chi tiết các công cụ tích hợp dữ liệu theo hướng tiếp cận “mashup”. Với mục đích và phạm vi của đề tài, sáu nhóm công cụ tích hợp dữ liệu chính được trình

bày bao gồm: (i) nhóm công cụ ABC trong phần 4.1, (ii) nhóm công cụ DEF trong phần 4.2, nhóm công cụ GHK trong phần 4.3, v.v...

Trong phần Kết chương, sinh viên đưa ra một số kết luận quan trọng của chương. Những vấn đề mở ra trong Tổng quan cần được tóm tắt lại nội dung và cách giải quyết/thực hiện như thế nào. Sinh viên lưu ý không viết Kết chương giống hệt Tổng quan. Sau khi đọc phần Kết chương, người đọc sẽ nắm được sơ bộ nội dung và giải pháp cho các vấn đề đã trình bày trong chương. Trong Kết chương, Sinh viên nên có thêm câu liên kết tới chương tiếp theo.

Ví dụ về phần Kết chương: Chương này đã phân tích chi tiết sáu nhóm công cụ tích hợp dữ liệu. Nhóm công cụ ABC và DEF thích hợp với những bài toán tích hợp dữ liệu phạm vi nhỏ. Trong khi đó, nhóm công cụ GHK lại chứng tỏ thế mạnh của mình với những bài toán cần độ chính xác cao, v.v. Từ kết quả nghiên cứu và phân tích về sáu nhóm công cụ tích hợp dữ liệu này, tôi đã thực hiện phát triển phần mềm tự động bóc tách và tích hợp dữ liệu sử dụng nhóm công cụ GHK. Phần này được trình bày trong chương tiếp theo – Chương 5.

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT

Chương này có độ dài không quá 10 trang. Chương này sinh viên trình bày về phân tích rõ ngữ cảnh bài toán cũng như các kết quả nghiên cứu tương tự. Đồng thời, sinh viên có thể trình bày thêm về các kiến thức nền tảng. Mỗi chương nên có đoạn mở đầu chương giới thiệu những nội dung sẽ trình bày trong chương.

2.1 Ngữ cảnh của bài toán

...

2.2 Các kết quả nghiên cứu tương tự

Trong phần này sinh viên trình bày các nghiên cứu liên quan (related work), chú ý phân tích rõ những ưu nhược điểm của chúng. Từ đó, nêu bật lên động lực để thực hiện nghiên cứu của đề án này.

2.3 Vòng đời của một gói tin trong Linux Networking Stack

Khi một gói tin đi vào hệ thống Linux, nó phải trải qua nhiều giai đoạn xử lý từ phần cứng đến nhân hệ điều hành trước khi được chuyển tới ứng dụng đích hoặc bị loại bỏ. Việc hiểu rõ vòng đời của gói tin là cơ sở để nghiên cứu các cơ chế xử lý mạng hiện đại như XDP, TC eBPF và Netfilter, bởi các công nghệ này đều can thiệp vào những vị trí khác nhau trong luồng xử lý của Linux Networking Stack.

Giai đoạn đầu tiên bắt đầu tại card mạng (Network Interface Card - NIC). Khi tín hiệu mạng được truyền qua môi trường vật lý như cáp Ethernet hoặc cáp quang, NIC sẽ tiếp nhận tín hiệu và chuyển đổi chúng thành dữ liệu số. Để giảm tải cho CPU, NIC sử dụng cơ chế Direct Memory Access (DMA) nhằm ghi trực tiếp dữ liệu của gói tin vào các vùng nhớ đã được cấp phát trước trong RAM. Các vùng nhớ này được tổ chức dưới dạng hàng đợi nhận (Receive Ring Buffer), cho phép NIC và CPU trao đổi dữ liệu với chi phí thấp hơn so với việc CPU phải tham gia trực tiếp vào quá trình sao chép dữ liệu [1].

Sau khi dữ liệu được ghi vào Receive Ring Buffer, NIC cần thông báo cho hệ điều hành rằng có gói tin mới cần được xử lý. Trong điều kiện lưu lượng thấp, NIC thường sử dụng cơ chế ngắt (interrupt) để báo hiệu cho CPU. Tuy nhiên, khi số lượng gói tin tăng cao, việc tạo ngắt cho từng gói tin sẽ gây ra hiện tượng interrupt storm, làm giảm hiệu năng hệ thống. Để giải quyết vấn đề này, Linux sử dụng cơ chế New API (NAPI). Khi lưu lượng tăng, driver NIC tạm thời vô hiệu hóa các ngắt nhận gói tin và chuyển sang chế độ polling. Thay vì xử lý từng ngắt riêng lẻ, CPU sẽ chủ động đọc nhiều gói tin từ Receive Ring Buffer trong một lần polling, giúp giảm số lần chuyển ngữ cảnh và nâng cao hiệu quả xử lý mạng.

Trong quá trình polling, driver NIC lấy các gói tin từ Receive Ring Buffer và chuyển chúng vào Linux Networking Stack. Tại đây, kernel tạo cấu trúc dữ liệu trung tâm gọi là socket buffer (struct sk_buff hay skb), đại diện cho gói tin trong suốt quá trình xử lý bên trong hệ điều hành. Ngoài dữ liệu thực tế của gói tin, skb còn chứa nhiều thông tin bổ sung (metadata) phục vụ cho các thành phần khác nhau của network stack [2].

Cấu trúc skb duy trì các offset hoặc con trỏ tới các lớp giao thức khác nhau của gói tin, bao gồm tầng liên kết dữ liệu (MAC layer), tầng mạng (Network layer) và tầng giao vận (Transport layer). Các trường như mac_header, network_header và transport_header cho phép kernel truy cập trực tiếp vào các phần header tương ứng mà không cần sao chép dữ liệu nhiều lần. Việc phân tích các trường header này được thực hiện dần bởi các thành phần của Linux Networking Stack khi gói tin đi qua các tầng xử lý khác nhau.

Đối với gói tin Ethernet, kernel xác định loại giao thức tầng trên thông qua trường EtherType. Nếu đây là gói tin IPv4 hoặc IPv6, gói tin sẽ được chuyển tới tầng mạng để xử lý tiếp. Tại tầng IP, kernel phân tích địa chỉ IP nguồn, địa chỉ IP đích và trường protocol nhằm xác định giao thức tầng giao vận tương ứng như TCP hoặc UDP. Sau đó, tầng giao vận tiếp tục xử lý các thông tin như cổng nguồn, cổng đích, số thứ tự gói tin và các cờ điều khiển của giao thức.

Sau khi đi qua các thành phần của network stack, gói tin tiếp tục được xử lý bởi các cơ chế như Traffic Control (TC), Netfilter, định tuyến (routing) và các bảng chuyển tiếp tương ứng. Dựa trên kết quả xử lý, gói tin có thể được chấp nhận, chuyển tiếp sang giao diện mạng khác hoặc bị loại bỏ.

Đối với các gói tin có đích là chính máy chủ, kernel sẽ thực hiện tra cứu socket dựa trên các thông tin đã phân tích từ header, đặc biệt là địa chỉ IP đích, giao thức và cổng đích. Nếu tìm thấy socket phù hợp, dữ liệu sẽ được đưa vào hàng đợi của socket đó để ứng dụng ở không gian người dùng (user space) có thể đọc và xử lý. Ngược lại, nếu không tồn tại socket tương ứng hoặc gói tin vi phạm các chính sách mạng của hệ thống, kernel sẽ loại bỏ gói tin hoặc phản hồi bằng các thông điệp lỗi thích hợp tùy theo giao thức được sử dụng.

Toàn bộ quá trình trên thể hiện kiến trúc phân tầng của Linux Networking Stack, trong đó mỗi thành phần đảm nhận một vai trò riêng nhằm cân bằng giữa hiệu năng, khả năng mở rộng và tính linh hoạt. Đây cũng là nền tảng để các công nghệ như XDP, eBPF và Netfilter can thiệp vào luồng xử lý mạng nhằm xây dựng các giải pháp firewall và giám sát lưu lượng hiệu năng cao.

2.4 eBPF

2.4.1 Lịch sử và tổng quan

eBPF (Extended Berkeley Packet Filter) là công nghệ cho phép thực thi các chương trình được kiểm chứng an toàn bên trong Linux kernel mà không cần sửa đổi mã nguồn kernel hoặc phát triển các Linux Kernel Modules (LKM). Nhờ cơ chế sandbox và bộ kiểm tra (verifier), eBPF cho phép mở rộng chức năng của kernel một cách linh hoạt trong khi vẫn đảm bảo tính ổn định của hệ thống.

Tiền thân của eBPF là Berkeley Packet Filter (cBPF), được Steven McCanne và Van Jacobson giới thiệu vào năm 1992 tại Lawrence Berkeley National Laboratory. Mục tiêu ban đầu của cBPF là tối ưu hóa quá trình lọc gói tin mạng bằng cách thực hiện việc kiểm tra gói tin ngay trong kernel trước khi chuyển dữ liệu lên không gian người dùng [3]. Công cụ tcpdump là một trong những ứng dụng tiêu biểu sử dụng cBPF.

Từ các phiên bản Linux kernel 3.x, đặc biệt từ Linux kernel 3.18 trở đi, cBPF được mở rộng thành eBPF bởi Alexei Starovoitov và Daniel Borkmann. eBPF sử dụng kiến trúc 64-bit với tập thanh ghi gồm 11 thanh ghi (R0-R10), cung cấp khả năng xử lý mạnh mẽ hơn đáng kể so với cBPF. Đồng thời, phạm vi ứng dụng của eBPF cũng được mở rộng từ lĩnh vực mạng sang nhiều lĩnh vực khác như giám sát hệ thống (observability), theo dõi thực thi (tracing), tối ưu hiệu năng và bảo mật hệ thống.

2.4.2 Cơ chế hoạt động và vòng đời của chương trình eBPF

Để một chương trình eBPF (Extended Berkeley Packet Filter) có thể hoạt động từ giai đoạn được viết ra cho đến khi chạy thực tế trong nhân hệ điều hành (kernel), nó phải trải qua một quy trình kiểm soát nghiêm ngặt và đa tầng. Quy trình này được thiết kế nhằm đảm bảo hai yếu tố cốt lõi: tốc độ thực thi tối ưu và tính an toàn tuyệt đối cho hệ thống, ngăn chặn mọi nguy cơ gây treo hoặc sụp đổ kernel.

Quá trình đầu tiên là Biên dịch (Compilation). Thay vì viết trực tiếp mã máy, lập trình viên thường phát triển mã nguồn eBPF bằng một tập con bị giới hạn của ngôn ngữ C (restricted C) hoặc các ngôn ngữ bậc cao khác như Rust, được hỗ trợ bởi eBPF. Mã nguồn này sau đó được biên dịch thông qua trình biên dịch Clang/LLVM, sử dụng backend eBPF để chuyển đổi thành mã bytecode eBPF trung gian, thường được đóng gói trong một file định dạng ELF (Executable and Linkable Format). File ELF này chứa cả mã bytecode và các thông tin siêu dữ liệu (metadata) cần thiết cho các bước tiếp theo.

Tiếp theo là giai đoạn tải vào Kernel (Loading). Chương trình bytecode cùng với

các cấu hình đi kèm sẽ được tải từ không gian người dùng (user-space) vào không gian kernel thông qua lệnh gọi hệ thống (system call) có tên là `bpf()`. Trong quá trình tải này, kernel cũng sẽ cấp phát và khởi tạo các cấu trúc dữ liệu đặc biệt gọi là "BPF Maps" (dạng key-value store), cho phép chương trình eBPF và ứng dụng user-space có thể trao đổi dữ liệu với nhau một cách hiệu quả.

Bước quan trọng nhất, được ví như "trái tim" tạo nên sự an toàn của eBPF, là quá trình Xác thực (Verification). Ngay khi bytecode được tải lên, kernel sẽ đưa nó qua một mô-đun gọi là eBPF Verifier (Bộ xác thực). Verifier sẽ thực hiện phân tích tĩnh sâu rộng, kiểm tra hàng ngàn nhánh điều kiện trong đồ thị luồng điều khiển (control flow graph) của chương trình. Cụ thể, Verifier đảm bảo rằng chương trình không chứa bất kỳ vòng lặp vô hạn nào (hoặc nếu có vòng lặp, chúng phải được giới hạn số lần lặp một cách chặt chẽ để tránh làm treo hoặc làm chậm kernel). Đồng thời, nó thực hiện kiểm tra ranh giới bộ nhớ (memory bounds checking) để đảm bảo chương trình không truy cập vào các vùng nhớ trái phép, và bắt buộc chương trình phải có điểm kết thúc rõ ràng, an toàn trong một khoảng thời gian xác định (deterministic execution time).

Nếu vượt qua được sự kiểm duyệt khắt khe của Verifier một cách hoàn hảo, mã bytecode sẽ bước vào giai đoạn Biên dịch JIT (Just-In-Time Compilation). Lúc này, trình biên dịch JIT tích hợp sẵn trong kernel sẽ dịch trực tiếp mã bytecode eBPF thành mã máy gốc (native machine code) tương thích chính xác với kiến trúc CPU của máy chủ đang chạy (ví dụ: x86-64, ARM64...). Việc biên dịch JIT này giúp loại bỏ hoàn toàn chi phí overhead của việc thông dịch (interpreting), cho phép chương trình eBPF chạy với tốc độ gần như tương đương với các hàm kernel được biên dịch sẵn từ trước.

Cuối cùng là giai đoạn Gắn vào Điểm nối (Hooking). Chương trình eBPF đã được biên dịch thành mã máy sẽ được gắn vào các "hook" (điểm bắt sự kiện) cụ thể trong kernel. Khi một sự kiện tương ứng xảy ra tại hook đó, chương trình eBPF sẽ lập tức được kích hoạt để xử lý. Các kiến trúc eBPF hiện đại hỗ trợ một hệ sinh thái hook vô cùng phong phú, bao gồm: Kprobes (để theo dõi các hàm nội bộ của kernel), Uprobes (theo dõi hàm trong không gian user-space), Tracepoints (các điểm theo dõi ổn định do kernel cung cấp), các lệnh gọi hệ thống (syscalls), và đặc biệt là các sự kiện mạng như XDP (eXpress Data Path) hoặc Traffic Control (TC). Tài liệu của Cilium cung cấp cái nhìn cực kỳ chi tiết về cách các hook mạng này tương tác với tập lệnh eBPF và cách sử dụng các thanh ghi (registers) để xử lý gói tin ở tốc độ cao.

2.4.3 BPF Maps

Các chương trình eBPF thường cần lưu trữ dữ liệu trạng thái hoặc trao đổi thông tin với các thành phần khác của hệ thống trong quá trình thực thi. Tuy nhiên, để đảm bảo tính an toàn và khả năng kiểm soát tài nguyên bên trong kernel, chương trình eBPF không được phép truy cập hoặc ghi dữ liệu một cách tùy ý vào bộ nhớ của hệ điều hành. Để giải quyết vấn đề này, Linux cung cấp cơ chế BPF Maps, đóng vai trò là vùng lưu trữ dữ liệu dùng chung giữa chương trình eBPF và các thành phần khác của hệ thống [4].

BPF Maps là các cấu trúc dữ liệu được quản lý bởi kernel và được tổ chức chủ yếu theo mô hình khóa/giá trị (key-value store). Chúng cho phép các chương trình eBPF lưu trữ thông tin trong quá trình xử lý và duy trì trạng thái giữa nhiều lần thực thi khác nhau. Đồng thời, các ứng dụng ở không gian người dùng (user space) cũng có thể truy cập các Maps thông qua lời gọi hệ thống `bpf()`, tạo nên cơ chế trao đổi dữ liệu hiệu quả giữa user space và kernel space [4].

Trong các hệ thống giám sát và bảo mật hiện đại, BPF Maps thường được sử dụng để lưu trữ các thông tin thống kê như số lượng gói tin được xử lý, số lượng kết nối đang hoạt động, độ trễ của các dịch vụ hoặc các sự kiện bất thường được phát hiện bởi chương trình eBPF. Dữ liệu này sau đó có thể được các ứng dụng user space đọc theo thời gian thực để phục vụ mục đích giám sát, trực quan hóa hoặc cảnh báo hệ thống [4].

Ngoài vai trò lưu trữ dữ liệu, BPF Maps còn cho phép truyền tham số cấu hình từ user space xuống chương trình eBPF đang chạy trong kernel. Nhờ đó, các chính sách xử lý có thể được thay đổi động mà không cần biên dịch hoặc nạp lại chương trình eBPF. Trong một số trường hợp, Maps còn được sử dụng để hỗ trợ cơ chế Tail Call, cho phép một chương trình eBPF chuyển việc thực thi sang chương trình eBPF khác nhằm xây dựng các pipeline xử lý phức tạp và có khả năng mở rộng cao [4].

Linux hiện hỗ trợ nhiều loại BPF Maps khác nhau nhằm đáp ứng các nhu cầu sử dụng đa dạng. Một trong những loại phổ biến nhất là `BPF_MAP_TYPE_HASH`, sử dụng cấu trúc bảng băm để lưu trữ dữ liệu theo cặp khóa/giá trị. Đối với các trường hợp cần truy cập dữ liệu thông qua chỉ số cố định với độ trễ thấp, `BPF_MAP_TYPE_ARRAY` là lựa chọn phù hợp. Để truyền luồng sự kiện từ kernel lên user space với hiệu năng cao, Linux cung cấp `BPF_MAP_TYPE_RINGBUF`, hoạt động theo cơ chế bộ đệm vòng (ring buffer). Ngoài ra, `BPF_MAP_TYPE_LRU_HASH` mở rộng từ Hash Map truyền thống bằng cách tự động loại bỏ các phần tử ít được sử dụng nhất khi đạt tới giới hạn bộ nhớ được cấu hình [4].

Nhờ cơ chế quản lý tập trung trong kernel cùng khả năng chia sẻ dữ liệu hiệu quả giữa nhiều thành phần của hệ thống, BPF Maps đã trở thành một trong những thành phần cốt lõi của kiến trúc eBPF, cho phép xây dựng các ứng dụng mạng, giám sát và bảo mật hiệu năng cao trên Linux [4].

2.5 XDP

2.5.1 Tổng quan và vị trí trong kernel

eXpress Data Path (XDP) là một framework xử lý gói tin hiệu năng cao được tích hợp trong Linux kernel, cho phép thực thi các chương trình eBPF tại vị trí rất sớm trong đường đi của gói tin mạng [1], [5]. Không giống các cơ chế xử lý truyền thống như Netfilter hay Traffic Control (TC), XDP được thực thi trực tiếp trong đường nhận gói tin (receive path) của driver mạng trước khi kernel cấp phát cấu trúc `sk_buff` (skb) và trước khi gói tin đi vào Linux Networking Stack truyền thống [2], [5].

Trong kiến trúc mạng thông thường của Linux, mỗi gói tin nhận được phải trải qua nhiều bước xử lý như cấp phát bộ nhớ cho cấu trúc skb, phân tích các lớp giao thức và chuyển tiếp qua các thành phần khác nhau của network stack [6]. Các bước này tạo ra chi phí xử lý đáng kể khi hệ thống phải xử lý lưu lượng mạng lớn. XDP được thiết kế nhằm giảm thiểu chi phí đó bằng cách cho phép kiểm tra, lọc hoặc chuyển hướng gói tin ngay tại tầng driver, từ đó loại bỏ các gói tin không mong muốn trước khi chúng tiêu tốn thêm tài nguyên hệ thống [5], [7].

Nhờ tận dụng cơ chế eBPF, các chương trình XDP có thể được nạp động vào kernel mà không cần sửa đổi mã nguồn kernel hoặc phát triển các kernel module riêng biệt. Đồng thời, chương trình vẫn phải vượt qua quá trình xác thực của eBPF Verifier nhằm đảm bảo tính an toàn trước khi được phép thực thi trong kernel [1], [4].

2.5.2 Các chế độ hoạt động

Linux hiện hỗ trợ ba chế độ triển khai XDP khác nhau tùy thuộc vào khả năng của phần cứng và trình điều khiển thiết bị mạng [5], [7].

Native Mode (Driver XDP) là chế độ hoạt động phổ biến và mang lại hiệu năng cao nhất. Trong chế độ này, chương trình XDP được thực thi trực tiếp trong đường nhận gói tin của driver mạng trước khi cấu trúc skb được tạo ra. Do đó, nhiều chi phí xử lý của Linux Networking Stack có thể được loại bỏ hoàn toàn [5].

Offloaded Mode cho phép chương trình XDP được thực thi trên các SmartNIC hoặc thiết bị mạng hỗ trợ XDP Offload. Khi đó, một phần hoặc toàn bộ việc xử lý gói tin có thể diễn ra ngay trên phần cứng mạng trước khi dữ liệu được chuyển vào

hệ thống chủ [5], [7].

Generic Mode (Generic XDP) được sử dụng khi driver mạng không hỗ trợ Native XDP. Trong trường hợp này, chương trình XDP được thực thi ở một vị trí muộn hơn trong Linux Networking Stack sau khi cấu trúc `skb` đã được cấp phát. Chế độ này cung cấp khả năng tương thích rộng nhưng hiệu năng thấp hơn so với Native Mode do không loại bỏ được chi phí tạo `skb` [2], [5].

2.5.3 Các return code

Một trong những đặc điểm quan trọng nhất của chương trình XDP là khả năng định đoạt số phận của gói tin ngay lập tức thông qua các mã trả về (return codes) sau khi kiểm tra [5], [7]. Các mã này được định nghĩa sẵn, bao gồm:

- **XDP_DROP**: Chỉ thị cho driver mạng loại bỏ gói tin ngay lập tức. Đây là mã trả về cốt lõi cho các ứng dụng tường lửa chống DDoS, giúp tiết kiệm CPU bằng cách vứt bỏ các gói tin ác ý trước khi chúng kịp phân bổ bất kỳ tài nguyên nào.
- **XDP_PASS**: Cho phép gói tin đi tiếp vào luồng xử lý mạng tiêu chuẩn (Linux Networking Stack). Gói tin sẽ được cấp phát cấu trúc `sk_buff` và đi qua các tầng như Netfilter/Iptables.
- **XDP_TX**: Chuyển tiếp (forward) gói tin ngược trở lại chính giao diện mạng (network interface) mà nó vừa đi vào, thường dùng trong cân bằng tải.
- **XDP_REDIRECT**: Chuyển hướng gói tin sang một giao diện mạng khác hoặc chuyển thẳng vào một socket (như `AF_XDP`) để đưa lên không gian người dùng xử lý (ví dụ: chuyển cho Suricata) mà không qua kernel stack.
- **XDP_ABORTED**: Biểu thị chương trình gặp lỗi trong quá trình thực thi. Gói tin bị loại bỏ, và kernel sẽ sinh ra một sự kiện (tracepoint) để phục vụ gỡ lỗi.

2.6 Firewall phân tầng

Firewall (tường lửa) là một thành phần thiết yếu trong kiến trúc bảo mật mạng, đóng vai trò như một chốt chặn (choke point) kiểm soát luồng giao thông vào ra giữa các mạng [8]. Sự phát triển của các cơ chế kiểm soát truy cập đã dẫn đến sự hình thành của các kỹ thuật lọc phân tầng, từ mức cơ bản ở tầng mạng (Network layer) đến mức phức tạp ở tầng ứng dụng (Application layer) theo mô hình tham chiếu OSI [9], [10].

2.6.1 Lọc gói tin tầng 3, tầng 4

Lọc gói tin phi trạng thái (Stateless Packet Filtering) là cơ chế kiểm soát truy cập cơ bản nhất, hoạt động chủ yếu tại tầng 3 (Network) và tầng 4 (Transport).

Firewall sẽ kiểm tra từng gói tin một cách độc lập dựa trên các thông tin tĩnh trong mào đầu (header) của gói tin, bao gồm địa chỉ IP nguồn, địa chỉ IP đích, giao thức sử dụng (TCP, UDP, ICMP) và cổng giao tiếp [8].

Cơ chế này so sánh thông tin gói tin với một tập hợp các quy tắc (ruleset) được định nghĩa sẵn. Nếu gói tin khớp với một quy tắc cho phép (allow/accept), nó sẽ được đi qua; ngược lại, nếu khớp với quy tắc từ chối (deny/drop), gói tin sẽ bị loại bỏ. Ưu điểm của lọc phi trạng thái là tốc độ xử lý nhanh, chi phí tài nguyên phần cứng thấp do không phải duy trì thông tin về các luồng kết nối [9]. Tuy nhiên, nhược điểm lớn nhất là nó không thể nhận biết được ngữ cảnh của gói tin, ví dụ như không thể phân biệt được một gói tin thuộc về một kết nối đã được thiết lập hợp lệ hay một gói tin giả mạo được gửi rời rạc [10].

2.6.2 Lọc trạng thái tầng 4

Để khắc phục điểm yếu của lọc phi trạng thái, cơ chế kiểm tra trạng thái (Stateful Packet Inspection - SPI) đã được phát triển. SPI không chỉ phân tích các thông tin tĩnh trên header mà còn duy trì một bảng trạng thái (state table) nhằm theo dõi vòng đời của tất cả các kết nối (connections) đang hoạt động đi qua firewall [10].

Khi một gói tin đầu tiên của một kết nối (ví dụ gói SYN trong TCP) đi tới, firewall sẽ kiểm tra nó dựa trên tập quy tắc tĩnh. Nếu được phép, một mục nhập (entry) mới sẽ được tạo trong bảng trạng thái lưu lại ngữ cảnh của luồng bao gồm cặp IP, cổng, giao thức và trạng thái kết nối hiện tại. Các gói tin tiếp theo thuộc cùng luồng này sẽ được kiểm tra chéo với bảng trạng thái, từ đó tối ưu hóa hiệu năng xử lý và tăng độ an toàn. Tính năng này giúp ngăn chặn hiệu quả các cuộc tấn công quét cổng lén lút (stealth scan) và chặn các gói tin phản hồi giả mạo không thuộc bất kỳ kết nối hợp lệ nào [9].

2.6.3 Kiểm tra gói tin tầng 7

Kiểm tra gói tin sâu (Deep Packet Inspection - DPI) mở rộng khả năng kiểm soát của firewall lên tầng ứng dụng (Layer 7). Khác với kỹ thuật SPI chỉ kiểm tra thông tin điều khiển ở phần header (tầng 3 và tầng 4), DPI tiến hành phân tích toàn bộ phần nội dung thực sự (payload) của gói tin để hiểu chính xác loại dữ liệu và giao thức ứng dụng đang được truyền tải (HTTP, DNS, FTP, v.v.) [10].

Bằng cách sử dụng các kỹ thuật đối sánh mẫu (pattern matching), phân tích hành vi hoặc giải mã giao thức, DPI có khả năng phát hiện phần mềm độc hại, kiểm soát quyền truy cập theo từng ứng dụng cụ thể và ngăn chặn các nguy cơ rò rỉ dữ liệu [10]. DPI cũng chính là thành phần cốt lõi của các tường lửa thế hệ mới (Next-Generation Firewall - NGFW), kết hợp khả năng kiểm soát truy cập truyền thống với tính năng phát hiện và ngăn chặn xâm nhập (IDS/IPS). Mặc dù đem lại

khả năng bảo vệ toàn diện, DPI đòi hỏi năng lực tính toán cực kỳ lớn và có thể gây ra độ trễ cao nếu không được tối ưu hóa bằng phần cứng hoặc sử dụng các cơ chế gia tốc xử lý luồng mạng chuyên dụng [8], [9].

2.7 Các dạng tấn công từ chối dịch vụ

Tấn công từ chối dịch vụ phân tán (Distributed Denial-of-Service - DDoS) là một trong những mối đe dọa nghiêm trọng nhất đối với tính sẵn sàng của các hệ thống mạng. Bản chất của DDoS là huy động một số lượng lớn các thiết bị bị thỏa hiệp (botnet) để đồng loạt gửi lưu lượng truy cập ác ý nhằm làm cạn kiệt tài nguyên của mục tiêu, khiến các người dùng hợp lệ không thể tiếp cận được dịch vụ [11]. Các phương thức tấn công có thể nhắm vào băng thông mạng, tài nguyên hệ điều hành, hoặc các giao thức tầng ứng dụng.

2.7.1 Tấn công SYN Flood

SYN Flood là một kiểu tấn công cạn kiệt tài nguyên (resource exhaustion) kinh điển nhắm vào quá trình bắt tay ba bước (three-way handshake) của giao thức TCP. Kẻ tấn công gửi liên tục hàng loạt các gói tin yêu cầu kết nối (TCP SYN) đến máy chủ mục tiêu với địa chỉ IP nguồn thường bị giả mạo [9], [11].

Khi nhận được gói SYN, máy chủ sẽ cấp phát tài nguyên trong cấu trúc dữ liệu (như hàng đợi kết nối nửa mở - half-open connection queue), phản hồi bằng gói SYN-ACK và chờ gói ACK cuối cùng từ client. Vì địa chỉ IP nguồn là giả mạo hoặc kẻ tấn công cố tình phớt lờ SYN-ACK, gói ACK cuối cùng sẽ không bao giờ được gửi đến. Kết quả là máy chủ duy trì trạng thái nửa mở cho đến khi hết thời gian chờ (timeout), làm cạn kiệt bảng trạng thái kết nối và từ chối các yêu cầu kết nối TCP hợp lệ sau đó [9].

2.7.2 Tấn công UDP Flood

UDP Flood là dạng tấn công làm cạn kiệt băng thông (volumetric attack) phổ biến nhất do đặc tính không trạng thái (stateless) và không yêu cầu thiết lập kết nối của giao thức UDP. Kẻ tấn công gửi một khối lượng khổng lồ các gói tin UDP chứa payload rác đến các cổng ngẫu nhiên trên máy chủ mục tiêu [11].

Khi máy chủ nhận được một gói tin UDP, nó phải kiểm tra xem có ứng dụng nào đang lắng nghe trên cổng đích hay không. Nếu không có ứng dụng nào, máy chủ phải trả lời bằng một thông điệp ICMP Destination Unreachable. Dưới áp lực của hàng triệu gói tin UDP mỗi giây, việc kiểm tra và phản hồi này sẽ làm cạn kiệt cả tài nguyên CPU lẫn băng thông mạng của nạn nhân. Hơn nữa, vì UDP không có cơ chế điều khiển tắc nghẽn (congestion control), nó rất dễ bị lạm dụng để tạo ra các luồng dữ liệu khổng lồ [9].

2.7.3 Tấn công HTTP Flood

Khác với SYN hay UDP Flood nhắm vào các tầng thấp, HTTP Flood là kiểu tấn công ở tầng ứng dụng (Layer 7). Trong dạng tấn công này, botnet được sử dụng để gửi các yêu cầu HTTP (thường là GET hoặc POST) dường như hoàn toàn hợp lệ đến máy chủ web nhằm làm quá tải năng lực xử lý [11].

Ví dụ, kẻ tấn công có thể yêu cầu tải liên tục các tệp tin có kích thước lớn hoặc liên tục truy vấn các endpoint yêu cầu máy chủ phải thực hiện truy vấn cơ sở dữ liệu phức tạp. Ưu điểm của HTTP Flood đối với kẻ tấn công là nó tiêu tốn ít băng thông hơn so với các cuộc tấn công volumetric nhưng lại gây ra tác động phá hoại lớn đến hệ thống đích, do việc xử lý một yêu cầu HTTP tốn nhiều tài nguyên CPU và RAM của máy chủ hơn rất nhiều so với việc sinh ra yêu cầu đó [12].

2.7.4 Slowloris

Slowloris là một dạng tấn công từ chối dịch vụ tốc độ thấp (low-and-slow DDoS) đặc biệt nguy hiểm nhắm vào các máy chủ web truyền thống phân bổ tài nguyên theo luồng (thread-based) như Apache. Khác với HTTP Flood dựa vào số lượng lớn yêu cầu, Slowloris hoạt động bằng cách mở nhiều kết nối hợp lệ đến máy chủ web và cố gắng giữ chúng mở càng lâu càng tốt [12].

Kẻ tấn công thực hiện điều này bằng cách gửi các yêu cầu HTTP không hoàn chỉnh (thiếu ký tự kết thúc header `\r\n\r\n`) và định kỳ gửi các HTTP header rác với tốc độ rất chậm để làm mới thời gian chờ (timeout) của máy chủ [12]. Do máy chủ web bắt buộc phải giữ luồng kết nối mở để chờ nhận toàn bộ yêu cầu, bảng giới hạn kết nối đồng thời (connection pool) sẽ nhanh chóng bị lấp đầy, ngăn chặn hoàn toàn việc phục vụ các kết nối mới từ người dùng hợp lệ.

2.7.5 IP Spoofing và Random Source Flooding

IP Spoofing (Giả mạo IP) là một kỹ thuật cốt lõi được sử dụng trong nhiều cuộc tấn công mạng, đặc biệt là DDoS. Kẻ tấn công sửa đổi địa chỉ IP nguồn trong tiêu đề gói tin IP thành một địa chỉ giả mạo. Điều này mang lại hai lợi thế lớn: thứ nhất, gây khó khăn cho việc truy vết nguồn gốc thực sự của lưu lượng tấn công; thứ hai, buộc các gói tin phản hồi (nếu có) từ nạn nhân phải gửi đến các địa chỉ không liên quan, tạo ra các cuộc tấn công khuếch đại phản xạ (Reflection Amplification Attack) [11].

Random Source Flooding là sự kết hợp của IP Spoofing, trong đó kẻ tấn công tạo ra một luồng gói tin ngập lụt với địa chỉ IP nguồn được tạo ngẫu nhiên (randomized) liên tục cho từng gói tin [11]. Hành vi này vô hiệu hóa các cơ chế bảo vệ dựa trên việc đếm tần suất kết nối hoặc chặn IP truyền thống (như Iptables), buộc các

firewall phải sử dụng các cơ chế phân tích nâng cao hoặc duy trì các bảng trạng thái khổng lồ, dẫn đến nguy cơ tự làm quá tải hệ thống bảo vệ.

2.8 Random Early Detection (RED) và Weighted RED (WRED)

2.8.1 Cơ chế Random Early Detection (RED)

Trong các thiết bị mạng truyền thống, cơ chế quản lý hàng đợi (Queue Management) phổ biến nhất là Tail Drop: khi hàng đợi (buffer) đầy, bất kỳ gói tin nào mới đến cũng sẽ bị loại bỏ. Tuy nhiên, cơ chế này gây ra hiện tượng đồng bộ hóa toàn cục (global synchronization), trong đó nhiều kết nối TCP cùng bị rút gói, cùng giảm tốc độ truyền (slow start), và cùng tăng tốc trở lại, dẫn đến tình trạng sử dụng băng thông không hiệu quả [13].

Để giải quyết vấn đề này, cơ chế Random Early Detection (RED) được Sally Floyd và Van Jacobson giới thiệu vào năm 1993 [13]. RED là một thuật toán quản lý hàng đợi chủ động (Active Queue Management - AQM). Thay vì đợi đến khi hàng đợi đầy, RED liên tục giám sát kích thước trung bình của hàng đợi. Khi kích thước trung bình vượt qua một ngưỡng tối thiểu (minimum threshold), thiết bị mạng bắt đầu chủ động loại bỏ (drop) gói tin một cách ngẫu nhiên với xác suất tăng dần theo kích thước hàng đợi [13], [14]. Việc loại bỏ gói tin ngẫu nhiên đóng vai trò như một tín hiệu cảnh báo sớm, thông báo cho các bên gửi TCP giảm tốc độ truyền trước khi tình trạng tắc nghẽn thực sự xảy ra, từ đó duy trì hiệu năng mạng ổn định.

2.8.2 Weighted RED và ưu tiên theo loại traffic

Weighted Random Early Detection (WRED) là một sự mở rộng của cơ chế RED, thường được triển khai trên các thiết bị định tuyến và tường lửa nhằm hỗ trợ Chất lượng dịch vụ (Quality of Service - QoS). Nếu như RED đối xử công bằng với tất cả các luồng giao thông thì WRED cho phép áp dụng các chính sách loại bỏ gói tin khác nhau dựa trên mức độ ưu tiên của dữ liệu [14].

WRED phân loại gói tin dựa trên các thông số như IP Precedence hoặc DSCP (Differentiated Services Code Point) trong phần header của gói tin IP. Mỗi nhóm phân loại sẽ được gán một mức trọng số (weight) với các thông số cấu hình RED riêng biệt (gồm ngưỡng tối thiểu, ngưỡng tối đa và tỷ lệ loại bỏ tối đa). Nhờ vậy, khi mạng bắt đầu có dấu hiệu tắc nghẽn, WRED sẽ ưu tiên loại bỏ các gói tin thuộc luồng dữ liệu ít quan trọng (ví dụ như luồng tải file) trước, trong khi cố gắng bảo vệ tối đa các gói tin thuộc luồng dữ liệu ưu tiên cao (như Voice over IP hoặc Video Streaming) khỏi việc bị loại bỏ [14].

2.8.3 Ứng dụng Probabilistic Drop trong môi trường DDoS

Mặc dù RED và WRED ban đầu được thiết kế để kiểm soát tắc nghẽn do lưu lượng hợp lệ gây ra, tư duy cốt lõi của chúng — loại bỏ gói tin dựa trên xác suất (Probabilistic Drop) — lại mang đến những ứng dụng đáng giá trong việc giảm thiểu (mitigation) các cuộc tấn công DDoS ngập lụt băng thông (Volumetric DDoS).

Trong bối cảnh bị tấn công DDoS cường độ cao, việc cố gắng xử lý và phân tích 100% lượng gói tin khổng lồ thường sẽ làm kiệt quệ tài nguyên xử lý (CPU/RAM) của hệ thống Firewall hoặc IPS. Khi áp dụng cơ chế Probabilistic Drop, hệ thống bảo vệ có thể thiết lập các chính sách chủ động loại bỏ ngẫu nhiên một phần lưu lượng ngay tại tầng mạng thấp khi nhận thấy băng thông hoặc mức độ sử dụng CPU vượt ngưỡng an toàn.

Hơn thế nữa, bằng cách kết hợp tư tưởng phân loại của WRED, hệ thống phòng thủ có thể tính toán "độ tin cậy" của các địa chỉ IP hoặc định danh luồng (flow ID). Các luồng kết nối có dấu hiệu bất thường (ví dụ: tần suất gửi cao đột biến) sẽ bị áp dụng xác suất rớt gói cao, trong khi các luồng từ người dùng đã xác thực hoặc có lịch sử truy cập tốt sẽ được bảo vệ với tỷ lệ rớt gói cực kỳ thấp. Điều này giúp hệ thống mạng vừa có khả năng sinh tồn qua các đợt DDoS khốc liệt, vừa duy trì được chất lượng dịch vụ (QoS) cao nhất có thể cho lượng tối đa khách hàng hợp lệ [13].

2.9 GeoIP, ASN và định dạng MMDB

2.9.1 Khái niệm GeoIP và ASN

GeoIP (Geolocation IP) là công nghệ cho phép xác định vị trí địa lý vật lý của một thiết bị kết nối Internet thông qua địa chỉ IP của nó. Thông tin GeoIP thường bao gồm quốc gia, thành phố, vĩ độ, kinh độ và mã vùng [15]. Trong an toàn thông tin, GeoIP đóng vai trò quan trọng để xây dựng các tập luật tường lửa dựa trên vị trí (Geo-blocking), giúp ngăn chặn các luồng truy cập bất thường đến từ các quốc gia không nằm trong tệp khách hàng hoặc là nguồn gốc thường xuyên của các mạng botnet.

Autonomous System Number (ASN) là một định danh duy nhất trên toàn cầu được gán cho một Hệ thống tự trị (Autonomous System - AS). Một AS là một nhóm các mạng IP được vận hành bởi một hoặc nhiều nhà khai thác mạng (như ISP, tập đoàn viễn thông) có chung một chính sách định tuyến thống nhất [9]. Việc nhận diện ASN giúp quản trị viên chặn toàn bộ dải IP của một nhà cung cấp đám mây hoặc một trung tâm dữ liệu cụ thể, những nơi thường bị lạm dụng để làm nguồn phát động tấn công DDoS.

2.9.2 Định dạng cơ sở dữ liệu MaxMind DB (MMDB)

MaxMind DB (MMDB) là một định dạng tệp cơ sở dữ liệu nhị phân chuyên dụng, được thiết kế tối ưu hóa cho tốc độ tra cứu (lookup) địa chỉ IP ở hiệu suất cực cao [15]. Cấu trúc của MMDB sử dụng cây tìm kiếm nhị phân (binary search tree), trong đó không gian địa chỉ IPv4 và IPv6 được phân nhánh thành các nút (nodes). Việc tìm kiếm một địa chỉ IP chỉ đòi hỏi duy nhất việc duyệt qua các bit của IP từ gốc cây xuống lá, mang lại độ phức tạp thời gian gần như tính ($O(1)$).

So với việc sử dụng các file CSV truyền thống đòi hỏi phân tích chuỗi (string parsing) đắt đỏ tốn nhiều tài nguyên CPU, MMDB mang lại tốc độ tra cứu đạt hàng triệu truy vấn mỗi giây. Điều này khiến nó trở thành định dạng tiêu chuẩn trong các hệ thống Firewall và IDS hiệu năng cao (như XDP và Suricata) phục vụ phân tích IP thời gian thực.

2.10 Iptables và Nftables

2.10.1 Kiến trúc Iptables và Netfilter

Iptables là công cụ quản trị tường lửa không gian người dùng (user-space) kinh điển trên Linux, hoạt động bằng cách giao tiếp trực tiếp với framework Netfilter bên trong kernel. Netfilter cung cấp một tập hợp các điểm neo (hooks) ở các vị trí khác nhau trong quá trình xử lý gói tin (như PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING) [6]. Khi một gói tin đi qua các hook này, Iptables áp dụng các tập quy tắc (rulesets) đã được định nghĩa trong các bảng (tables) và chuỗi (chains) để quyết định số phận của gói tin (ACCEPT, DROP, REJECT). Tuy nhiên, kiến trúc của Iptables gặp phải hạn chế nghiêm trọng về mặt hiệu năng khi số lượng quy tắc tăng lên hàng vạn, do cơ chế duyệt quy tắc tuần tự (linear processing) gây ra độ trễ quá lớn [6].

2.10.2 Sự chuyển dịch sang Nftables

Nftables được phát triển để thay thế Iptables, giải quyết các hạn chế về cấu trúc cứng nhắc và hiệu năng. Nftables hợp nhất các công cụ riêng lẻ (iptables, ip6tables, arptables, ebtables) thành một framework duy nhất. Điểm khác biệt cốt lõi là Nftables sử dụng một máy ảo (pseudo-state machine) bên trong kernel. Thay vì kernel phải chứa mã nguồn để hiểu từng giao thức một, Nftables biên dịch các quy tắc từ user-space thành một tập lệnh bytecode rút gọn, sau đó kernel chỉ việc thực thi bytecode đó. Đặc biệt, Nftables hỗ trợ kiến trúc dữ liệu Set và Map, cho phép tra cứu hàng vạn địa chỉ IP với độ trễ logarit thay vì tuyến tính, tối ưu hóa triệt để tường lửa trong môi trường mạng quy mô lớn.

2.11 Suricata

2.11.1 Tổng quan về Suricata

Suricata là hệ thống phân tích mạng, phát hiện và ngăn chặn xâm nhập (IDS/IPS) mã nguồn mở mạnh mẽ, được vận hành bởi Open Information Security Foundation (OISF). Sự khác biệt lớn nhất của Suricata so với công cụ đời đầu như Snort nằm ở kiến trúc đa luồng (multi-threaded) gốc. Kiến trúc này cho phép Suricata khai thác trọn vẹn sức mạnh xử lý của các hệ thống CPU đa nhân hiện đại, phân bổ xử lý luồng mạng lên nhiều lõi khác nhau để đáp ứng băng thông giám sát lên tới hàng chục Gbps [16].

2.11.2 Vai trò trong kiểm tra gói tin sâu (DPI)

Với tư cách là một engine phân tích ở tầng ứng dụng (DPI), Suricata có khả năng lắp ráp lại (reassemble) các luồng TCP, giải mã và phân tích nội dung của hàng loạt giao thức phức tạp (HTTP, TLS, DNS, SMB) [16]. Trong chế độ ngăn chặn chủ động (IPS), Suricata tích hợp chặt chẽ với Netfilter (qua NFQUEUE) hoặc eBPF/XDP để ra lệnh DROP lập tức các gói tin mang theo chữ ký tấn công (signature), chẳng hạn như mã độc, khai thác SQL hay các luồng HTTP Flood tinh vi nguy trang thành người dùng hợp pháp.

2.12 Connection Tracking (Conntrack)

2.12.1 Nguyên lý hoạt động

Connection Tracking (Conntrack) là mô-đun cốt lõi của Linux Netfilter, cung cấp khả năng tường lửa trạng thái (Stateful Firewall) [17]. Để theo dõi vòng đời kết nối, Conntrack trích xuất bộ tham số (tuple) từ gói tin bao gồm: IP nguồn, IP đích, Cổng nguồn, Cổng đích và Giao thức L4. Sử dụng bộ tuple này, Conntrack khởi tạo và duy trì một mục (entry) trong bảng băm (hash table) của kernel để theo dõi các trạng thái của kết nối như NEW, ESTABLISHED, RELATED hoặc INVALID [6], [17].

2.12.2 Conntrack trong các hệ thống hiệu năng cao

Conntrack là xương sống của cơ chế NAT và bảo mật Linux. Khi một luồng gói tin đã đạt trạng thái ESTABLISHED, firewall có thể sử dụng cờ này để bỏ qua việc quét lại toàn bộ bộ luật phức tạp và tự động cho phép gói tin đi qua, giúp giảm thiểu tải CPU. Tuy nhiên, trong các đợt DDoS ngập lụt, đặc biệt là IP Spoofing, bảng hash table của Conntrack rất dễ bị lấp đầy (trình trạng "conntrack table full") [17]. Lúc này, kernel sẽ buộc phải vứt bỏ các gói tin mới đến do không còn không gian bộ nhớ để lưu trạng thái. Vì thế, việc tinh chỉnh timeout và cấu hình giới hạn bảng Conntrack, kết hợp thả lỏng (stateless) cho các luồng không cần thiết là kỹ thuật

sinh tồn bắt buộc cho các máy chủ đối mặt với DDoS.

2.13 Tổng kết chương

Chương này đã giới thiệu toàn diện những cơ sở lý thuyết tạo nên một hệ thống bảo vệ mạng hiện đại. Quá trình di chuyển của gói tin qua Linux Networking Stack, sức mạnh mở rộng của eBPF và XDP, cùng kiến trúc của các hệ thống như Netfilter, Iptables/Nftables và Suricata đã làm rõ bức tranh xử lý và phân tích dữ liệu ở cả không gian nhân (kernel) lẫn ứng dụng. Sự hiểu biết sâu sắc về các chiến lược tấn công từ chối dịch vụ (DDoS) và những cơ chế phòng thủ chủ động như Stateful Firewall, Geo-blocking qua MMDB hay quản lý hàng đợi WRED cung cấp các luận cứ khoa học quan trọng. Những nền tảng lý thuyết này sẽ là tiền đề vững chắc phục vụ trực tiếp cho quá trình phân tích thiết kế, cài đặt và đánh giá hiệu năng kiến trúc tường lửa phân tầng trong các chương tiếp theo của đồ án.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

3.1 Tổng quan giải pháp

Phần này mô tả tổng quan giải pháp gồm những bước chính nào, hoạt động ra sao. Mục tiêu của phần này là cho người đọc một cái nhìn tổng thể về giải pháp. Để cho dễ hiểu thì nên có một biểu đồ mô tả luồng hoạt động của giải pháp đề xuất.

3.2 Phát biểu bài toán

3.3 Kiến trúc Hybrid Firewall tổng thể

3.4 Luồng xử lý gói tin

3.5 Thiết kế Tier 1 — Adaptive Mitigation

3.5.1 Tổng quan Tier 1

3.5.2 geo_trust_map

3.5.3 trusted_map

3.5.4 mitigation_map

3.5.5 Cơ chế Probabilistic Drop

3.6 Thiết kế Tier 2 — Per-IP Defense

3.6.1 Tổng quan Tier 2

3.6.2 rate_limit_map

3.6.3 auto_block_map

3.6.4 subnet_map và rule_map

3.7 Watcher Daemon và cơ chế Adaptive

3.7.1 Kiến trúc Watcher

3.7.2 Telemetry và Sliding Window

3.7.3 Hysteresis — Escalation và De-escalation

3.7.4 Tích hợp tín hiệu từ Suricata và Iptables

3.7.5 Cơ chế Manual Override

3.8 Control Plane

3.8.1 Go Controller

3.8.2 Conntrack Listener và trusted_map

3.8.3 Admin Seed và geo_trust_map

3.8.4 Thiết kế REST API

3.9 Nguyên tắc thiết kế

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

4.1 Môi trường triển khai

4.1.1 Cấu hình phần cứng

4.1.2 Cấu hình phần mềm

4.1.3 Mô hình máy ảo

4.2 Cấu trúc mã nguồn

4.2.1 Tổng quan cấu trúc thư mục

4.2.2 Phân kế thừa và phần mở rộng

4.3 Các thành phần chính

4.3.1 XDP Program

4.3.2 Go Controller

4.3.3 REST API

4.3.4 Web UI

4.4 Các chức năng chính

4.4.1 Quản lý Rule

4.4.2 Quản lý Tier 1

4.4.3 Quản lý Tier 2

4.4.4 Theo dõi Telemetry

4.5 Kết quả triển khai

CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 Mô hình thực nghiệm

5.1.1 Mô hình hệ thống

5.1.2 Công cụ và phương pháp đo lường

5.1.3 Các tham số đánh giá

5.2 Kịch bản Baseline

5.3 Kịch bản SYN Flood với IP cố định

5.4 Kịch bản Random Source Flood

5.5 Kịch bản HTTP Application Layer Attack

5.6 Phân tích kết quả

5.6.1 Hiệu quả của Tier 1 với Random Source Flood

5.6.2 Hiệu quả của Tier 2 với IP cố định

5.6.3 Vai trò của Geo Trust

5.6.4 Vai trò của Suricata và Iptables

5.7 Hạn chế của hệ thống

CHƯƠNG 6. KẾT LUẬN

6.1 Kết luận

Sinh viên nhắc lại các vấn đề mà đồ án đã giải quyết được, cũng như những vấn đề còn tồn đọng của đồ án.

6.2 Hướng phát triển trong tương lai

Sinh viên đề xuất hướng phát triển trong tương lai (nếu có) .

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

hovy1993automated E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

peterson2007computer L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

NguyenThucHai N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

poesio2001discourse M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

knott1996data A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

BernersTim T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z> (visited on 09/30/2010).

LectureA Princeton University, *Wordnet*. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml> (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] K. Castillon du Perron, “Accelerating Container Networking with eBPF: Performance Characterization, Evaluation, and Optimization,” phdthesis, Universite Cote d’Azur, 2025. **urlseen 10 june 2026.** **url:** <https://theses.hal.science/tel-05467830v1/file/2025COAZ4024.pdf>
- [2] Linux Kernel Documentation, *struct sk_buff Documentation*, 2025. **urlseen 10 june 2026.** **url:** <https://docs.kernel.org/networking/skbuff.html>
- [3] S. McCanne **and** V. Jacobson, “The BSD Packet Filter: A New Architecture for User-level Packet Capture,” *in Proceedings of the USENIX Winter 1993 Conference* USENIX Association, 1993. **url:** <https://www.tcpdump.org/papers/bpf-usenix93.pdf>
- [4] Linux Kernel Documentation, *BPF Documentation*, 2025. **urlseen 10 june 2026.** **url:** <https://docs.kernel.org/bpf/>
- [5] Linux Kernel Documentation, *eXpress Data Path (XDP)*, 2025. **urlseen 10 june 2026.** **url:** <https://docs.kernel.org/networking/xdp-rx-metadata.html>
- [6] C. Benvenuti, *Understanding Linux Network Internals*. O’Reilly Media, 2005.
- [7] Cilium Project, *BPF and XDP Reference Guide*, 2025. **urlseen 10 june 2026.** **url:** <https://docs.cilium.io/en/stable/bpf/>
- [8] W. R. Cheswick, S. M. Bellovin **and** A. D. Rubin, *Firewalls and internet security: repelling the wily hacker*. Addison-Wesley Professional, 2003.
- [9] J. F. Kurose **and** K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th. Pearson, 2017.
- [10] K. Scarfone **and** P. Hoffman, “Guidelines on firewalls and firewall policy,” National Institute of Standards **and** Technology, techreport NIST Special Publication 800-41 Revision 1, 2009.
- [11] J. Mirkovic **and** P. Reiher, “A taxonomy of DDoS attack and DDoS defense mechanisms,” *ACM SIGCOMM Computer Communication Review*, **jourvol 34, number 2, pages 39–53**, 2004.
- [12] E. Cambiaso, G. Papaleo, G. Chiola **and** M. Aiello, “Slow DoS attacks: definition and categorisation,” *International Journal of Trust Management in Computing and Communications*, **jourvol 1, number 3-4, pages 300–319**, 2013.

- [13] S. Floyd **and** V. Jacobson, “Random early detection gateways for congestion avoidance,” *IEEE/ACM Transactions on networking*, **jourvol** 1, **number** 4, **pages** 397–413, 1993.
- [14] B. Braden **and others**, “Recommendations on queue management and congestion avoidance in the internet,” RFC 2309, April, techreport, 1998.
- [15] MaxMind, *MaxMind DB File Format Specification*, 2025. **urlseen** 10 **june** 2026. **url**: <https://maxmind.github.io/MaxMind-DB/>
- [16] OISF, *Suricata User Guide*, 2025. **urlseen** 10 **june** 2026. **url**: <https://suricata.readthedocs.io/>
- [17] P. N. Ayuso, “Netfilter’s connection tracking system,” *login: the USENIX magazine*, **jourvol** 31, **number** 3, **pages** 34–39, 2006.

PHỤ LỤC

A. HƯỚNG DẪN VIẾT DATN

A.1 Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template DATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ DATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong DATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày DATN theo kiểu viết ý hoặc gạch đầu dòng. DATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết DATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.2 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
 - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
 - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
 - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - Chương trình ICT Global: Ngành Information Technology
 - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.3 Đánh dấu (bullet) và đánh số (numering)

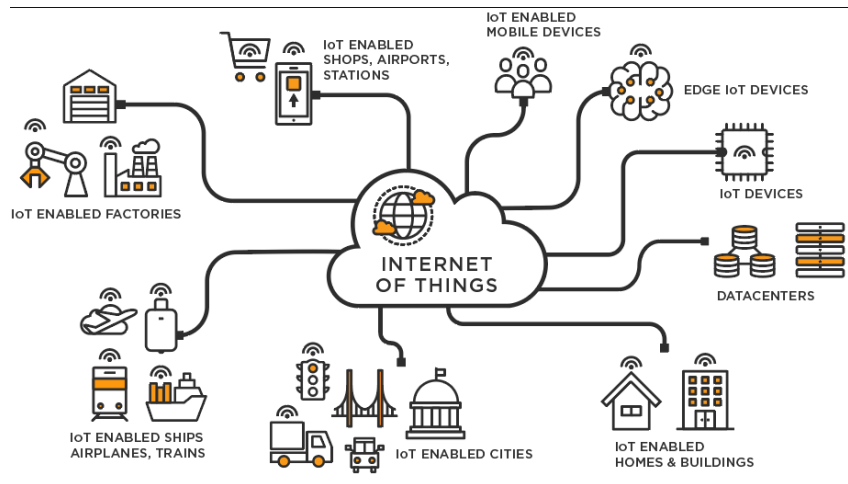
Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```



Hình A.1: Internet vạn vật

`\end{enumerate}`

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.4 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.5 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.6 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau **scott2013sdn**. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau **ashton2009internet**. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.7 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.8 Qui cách đóng quyển

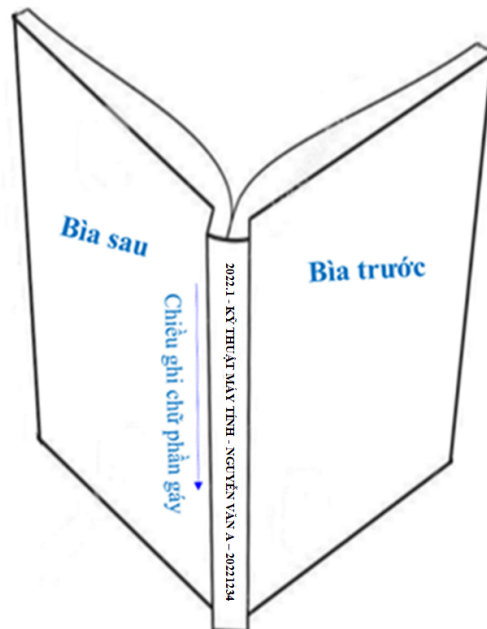
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐÁTN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐÁTN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Thông kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...